Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Курс Jetpack Compose



Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 20. Hilt. Custom компонент

17 декабря 2021

В этом уроке создаем кастомный компонент

Если нас по каким-то причинам не устраивают стандартные Hilt компоненты, мы можем создать свой. Обычно это необходимо, если надо компонент привязать к времени жизни, отличному от Lifecycle стандартных Android сущностей типа Activity, Fragment и т.п.

Рассмотрим пример.

Есть некий репозиторий MyRepository. И есть фрагмент RootFragment. В нем мы планируем открывать несколько дочерних фрагментов. Для всех этих фрагментов мы хотели бы иметь общий Singleton репозиторий с тем же временем жизни, что и у RootFragment.

Т.е. зашли на Root экран, создался репозиторий и используется всеми дочерними фрагментами. Повернули экран, репозиторий продолжает жить. Ушли с Root экрана - репозиторий больше не нужен. Когда снова открываем экран Root - создается новый Singleton экземпляр репозитория.

Обычно, если нужен Singleton, то первое, что приходит в голову - это просто повесить на объект Singleton аннотацию. Синглтон этого объекта будет жить в App компоненте и все фрагменты будут работать с этим *синглтоном*.

Но нам надо, чтобы репозиторий жил не все время работы приложения, а только на время жизни Root фрагмента. Поэтому этот классический вариант нам не подходит.

Реализуем это с помощью кастомного компонента.

Создаем компонент

```
@DefineComponent(parent = SingletonComponent::class)
interface MyComponent {

}
```

В качестве родителя необходимо указать компонент. Наш кастомный компонент станет сабкомпонентом этого родителя.

Обратите внимание, что мы в компоненте не описали никаких методов. Потому что это делается с помощью EntryPoint:

```
@InstallIn(MyComponent::class)
@EntryPoint
interface MyEntryPoint {
    fun getMyRepository(): MyRepository
}
```

Создаем EntryPoint для нашего компонента и просим его возвращать нам репозиторий.

Осталось создать билдер компонента. Тут все стандартно.

```
@DefineComponent.Builder
interface MyComponentBuilder {
    fun build(): MyComponent
}
```

Компонент готов, он умеет создавать репозиторий. Теперь надо добавить ему Scope, чтобы он держал синглтон репозитория, а не создавал новый экземпляр каждый раз.

Создаем свою Scope аннотацию:

```
@Scope
@MustBeDocumented
@kotlin.annotation.Retention(AnnotationRetention.RUNTIME)
public annotation class MyScope {}
```

Вешаем ее на компонент:

```
@MyScope
@DefineComponent(parent = SingletonComponent::class)
interface MyComponent {

}
```

И на репозиторий:

```
@MyScope
class MyRepository @Inject constructor() {

}
```

Теперь компонент будет держать синглтон репозитория.

Дальше нужно решить вопрос с тем, как передавать компонент во все фрагменты, чтобы они могли из него получить синглтон репозитория.

Просто инджектить билдер компонента во все фрагменты - не вариант. Потому что эти билдеры будут создавать разные экземпляры компонента. А разные экземпляры компонента будут давать разные синглтоны.

Нам нужен объект, который будет отвечать за создание, хранение и время жизни компонента.

Давайте создадим такой объект:

```
@Singleton
class MyComponentManager @Inject constructor(
    val myComponentBuilder: MyComponentBuilder
) {

    private var myComponent: MyComponent? = null

    fun create() {
        myComponent = myComponentBuilder.build()
    }

    fun get() = myComponent

    fun destroy() {
        myComponent = null
    }

}
```

Это объект со scope Singleton. Т.е. он живет, пока живет приложение. Мы будем инджектить этот объект в Root фрагмент и в те фрагменты, которым нужен будет компонент.

На вход он получает билдер компонента.

Метод create создает компонент. Этот метод нам надо будет вызывать, когда мы открываем Root фрагмент. Тем самым мы начнем жизненный цикл компонента.

Методом get любой желающий сможет получить компонент. Все будут получать один и тот же экземпляр.

Метод destroy надо будет вызвать, когда мы уходим с фрагмента Root. Тем самым мы заканчиваем жизненный цикл компонента. Сборщик мусора уничтожит компонент и его синглтоны, т.е. репозиторий.

Добавляем инджект этого объекта в Root фрагмент и вызываем методы жизненного цикла.

```
@AndroidEntryPoint
class RootFragment : Fragment() {

    @Inject
    lateinit var myComponentManager: MyComponentManager

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        if (savedInstanceState == null) {
            myComponentManager.create()
        }
    }

    override fun onDestroy() {
        super.onDestroy()
        if (isFinishing()) {
            myComponentManager.destroy()
        }
    }

    // ...

}
```

В onCreate мы создаем компонент, а в onDestroy - уничтожаем. Обратите внимание, что это не должно происходить при поворотах экрана. Нам надо, чтобы компонент мог их пережить.

Метода isFinishing во фрагменте не существует, к сожалению. Его реализацию придется сделать самим. Как вариант, можно к фрагменту привязать пустую ViewModel, которая в своем onCleared методе будет ставить какой-то флаг. Фрагмент читает этот флаг и понимает, что его работа тоже завершается.

Теперь время жизни компонента равно времени жизни фрагмента Root. Осталось использовать компонент в дочерних фрагментах:

```
@AndroidEntryPoint
class SomeFragment : Fragment() {

    @Inject
    lateinit var myComponentManager: MyComponentManager

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val myComponent = myComponentManager.get()
        val repository = EntryPoints.get(myComponent, MyEntryPoint::class.java).getMyRepository
    }
}
```

Инджектим MyComponentManager и используем метод get, чтобы получить компонент. А чтобы получить репозиторий, необходимо использовать EntryPoint.

Возможно, возникает вопрос: какой смысл городить все это, если все равно приходится создавать объект, который будет иметь Singleton scope и будет жить, пока живет приложение. Проще было сразу репозиторий сделать с этим scope и все.

Разница в том, что репозиторий может быть достаточно тяжелым объектом, который содержит в себе кучу других объектов. И держать все это в памяти все время работы приложения не имеет смысла. Да и кроме репозитория нам могут понадобиться еще объекты.

А объект MyComponentManager - легкий. Он создает и держит компонент (и его синглтон объекты) только когда нам это нужно.

Назад Вперёд

Комментарии

Вопрос по тексту — Сергей 10.04.2022 11:19
В тексте урока указано следующее:
"Компонент готов, он умеет создавать репозиторий. Теперь надо добавить ему Scope, чтобы он держал синглтон компонента, а не создавал новый экземпляр каждый раз."
Возможно я ошибаюсь, но может речь идёт не о синглтоне компонента, а о синглтоне репозитория?
Ответить

RE: Вопрос по тексту — Dmitry Vinogradov 22.04.2022 14:26
Верно, спасибо!
Ответить

альтернативное решение — Марина 01.08.2023 16:57
Можно ли решить эту задачу, используя Scope от ViewModel RootFragment, который предоставляет Hilt, для хранения экземпляра репозитория?
Ответить

RE: альтернативное решение — Dmitry Vinogradov 10.08.2023 16:43
Для RootFragment это сработает, но надо будет придумывать что-то для передачи репы в дочерние фрагменты
Ответить

Зачем использовать Builder? — Даниил 06.01.2024 15:52
А зачем нам тут Builder, если мы никаких модулей, созданных вручную, не передаем? Почему нельзя в MyComponentManager инжектить MyComponent напрямую?
Ответить

RE: Зачем использовать Builder? — Dmitry Vinogradov 12.01.2024 20:22
Чтобы иметь возможность пересоздавать компонент. Вызов в destroy() мы его уничтожаем.
А MyComponentManager - это штука, которая живет долго. Мы в нем будем несколько раз пересоздавать компонент, пока живет приложение.
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

jComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.